OLYMPIC HISTORY DATABASE

by

Nate Wright

IS 4420

Fall 2025

Abstract

This project created a comprehensive MySQL relational database from over 271,000 Olympic participation records spanning 1896-2016. It utilizes six normalized tables with enforced primary and foreign key constraints to model complex relationships. The project included systematic ETL, data cleaning, standardization, advanced SQL query development, and user privilege management, demonstrating proficiency in all database phases. Analytical queries revealed key insights into medal distribution and athlete demographics. The final system adheres to normalization and referential integrity and implements efficient data security measures.

Data Selection

Dataset: *120 years of Olympic history: athletes and results*

- Data on Olympic athlete bios and medal counts from every Olympic Games from 1896 to 2016
- This complete dataset will enable the creation of a relational database that explores athlete performances and provides athlete and event information for every Olympic Games. This will provide insights into performance by country, event, or year throughout all of Olympic history, creating an opportunity for users to analyze relationships and trends within the Olympics.

User Requirements and Business Rules

User Requirements

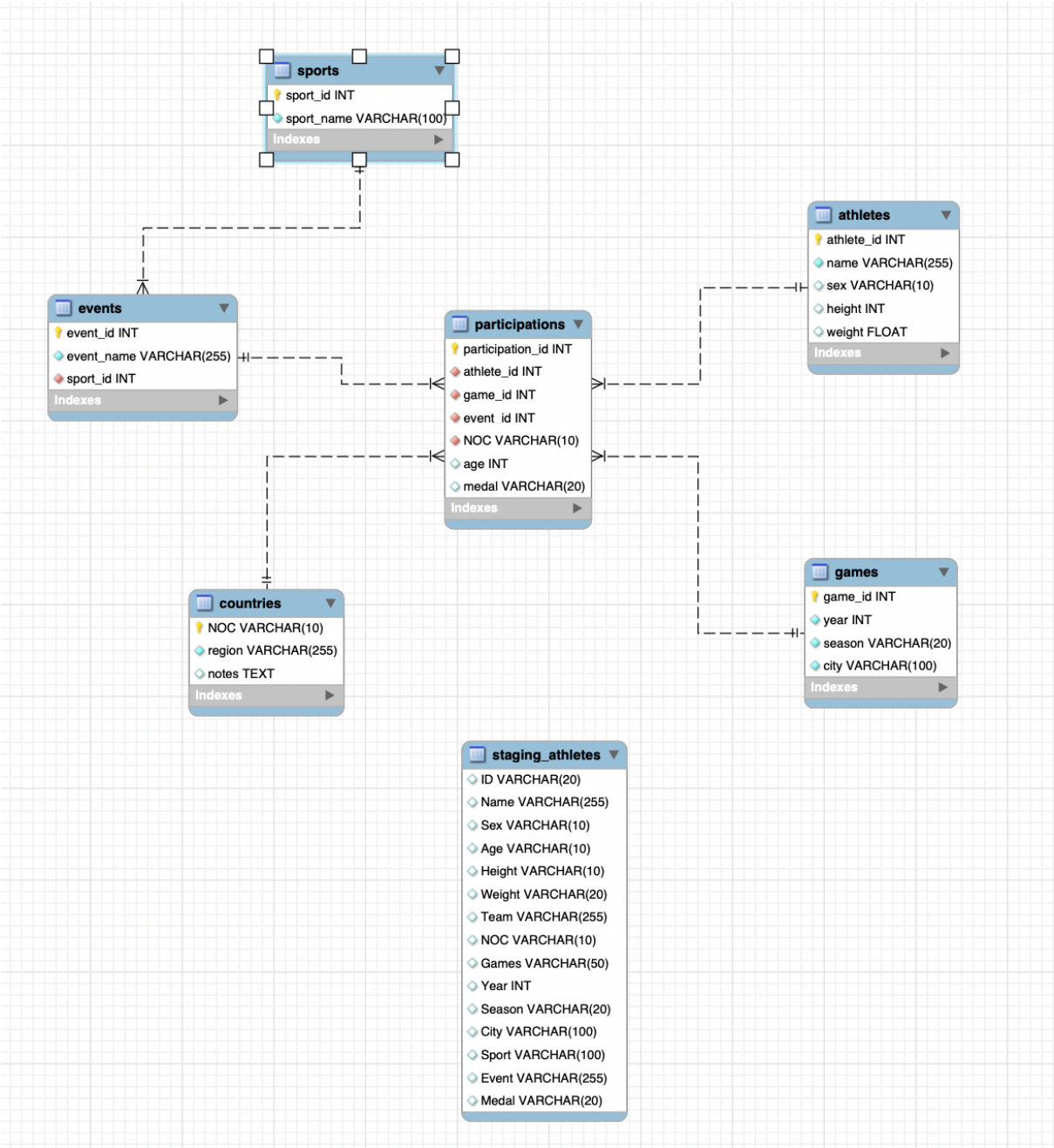
- The system shall allow users to track athlete participation across multiple Olympic Games and events
- The system shall enable users to analyze medal counts by country, sport, year, and season
- The system shall maintain records of all athlete demographics, including height, weight, age, and gender.

Business Rules

- Each participation record must have a valid athlete, game, event, and country
- Each athlete can participate in many events across multiple games, but each participation record belongs to exactly one athlete
- Medal values must be ‘'Gold', 'Silver', 'Bronze', or NULL (no medal) only.

- Each country can have many participations, but each participation record belongs to one country.
- Athletes must have a unique athlete_id, and athlete names are required but may have duplicates.

ERD



ETL and EDA Documentation

- Import Olympics CSV to staging table
- Clean 'NA' values to NULL for easy import

```sql
-- Convert all 'NA' strings to NULL in staging table
UPDATE staging_athletes
SET
    Age = NULLIF(Age, 'NA'),
    Height = NULLIF(Height, 'NA'),
    Weight = NULLIF(Weight, 'NA'),
    Medal = NULLIF(Medal, 'NA');
```

- Create dimension tables with clean INSERTs

```sql
-- Populate from staging
INSERT INTO athletes (athlete_id, name, sex, height, weight)
SELECT DISTINCT
    CAST(ID AS UNSIGNED),
    Name,
    Sex,
    CAST(Height AS UNSIGNED),
    CAST(Weight AS DECIMAL(6,2))
FROM staging_athletes
ORDER BY CAST(ID AS UNSIGNED);
```

- Create participations table

```sql
-- Create participations table
CREATE TABLE participations (
    participation_id INT AUTO_INCREMENT PRIMARY KEY,
    athlete_id INT NOT NULL,
    game_id INT NOT NULL,
    event_id INT NOT NULL,
    NOC VARCHAR(10) NOT NULL,
    age INT,
    medal VARCHAR(20),
    FOREIGN KEY (athlete_id) REFERENCES athletes(athlete_id),
    FOREIGN KEY (game_id) REFERENCES games(game_id),
    FOREIGN KEY (event_id) REFERENCES events(event_id),
    FOREIGN KEY (NOC) REFERENCES countries(NOC),
    CHECK (medal IS NULL OR medal IN ('Gold', 'Silver', 'Bronze'))
);
```

- During the ETL process, I encountered data quality issues in which missing values were represented as NA rather than NULL. To handle this, I initially loaded all data into a staging table with VARCHAR data types, then performed transformations to convert NA values to proper NULL values and cast numeric columns to their appropriate data types.
- “The athletes table uses athlete_id as a primary key without AUTO_INCREMENT to maintain the original dataset's ID structure. When inserting new athletes, I used next available ID using MAX(athlete_id) + 1 to ensure uniqueness while preserving integrity with the participations table."

SQL Code

SELECT Statement Examples

```sql
-- 1. Total medals awarded in Summer Olympics (by country)
SELECT
    c.region AS country,
    COUNT(p.medal) AS total_summer_medals,
    SUM(CASE WHEN p.medal = 'Gold' THEN 1 ELSE 0 END) AS gold,
    SUM(CASE WHEN p.medal = 'Silver' THEN 1 ELSE 0 END) AS silver,
    SUM(CASE WHEN p.medal = 'Bronze' THEN 1 ELSE 0 END) AS bronze
FROM participations p
JOIN countries c ON p.NOC = c.NOC
JOIN games g ON p.game_id = g.game_id
WHERE p.medal IS NOT NULL
  AND g.season = 'Summer'
GROUP BY c.region
ORDER BY total_summer_medals DESC
LIMIT 20;
```

country	total_summer_med...	gold	silver	bronze
United States of America	4836	2377	1304	1155
Russia	3188	1220	974	994
Germany	3122	1074	986	1062
UK	1983	635	728	620
France	1627	465	575	587
Italy	1446	518	474	454
Australia	1333	362	456	515
Hungary	1123	432	328	363
Sweden	1106	352	396	358
China	913	335	319	259
Netherlands	906	238	298	370
Japan	850	230	287	333

Result 34

```sql
-- 3. Top 10 athletes by total medal count
SELECT
    a.name,
    c.region AS country,
    COUNT(p.medal) AS total_medals,
    SUM(CASE WHEN p.medal = 'Gold' THEN 1 ELSE 0 END) AS gold,
    SUM(CASE WHEN p.medal = 'Silver' THEN 1 ELSE 0 END) AS silver,
    SUM(CASE WHEN p.medal = 'Bronze' THEN 1 ELSE 0 END) AS bronze
FROM participations p
JOIN athletes a ON p.athlete_id = a.athlete_id
JOIN countries c ON p.NOC = c.NOC
WHERE p.medal IS NOT NULL
GROUP BY a.name, c.region
ORDER BY total_medals DESC, gold DESC
LIMIT 10;
```

name	country	total_medals	gold	silver	bronze
Michael Fred Phelps, II	United States of America	28	23	3	2
Larysa Semenivna Latynina (Diriy-)	Russia	18	9	5	4
Nikolay Yefimovich Andrianov	Russia	15	7	5	3
Ole Einar Bjrndalen	Norway	13	8	4	1
Borys Anfiyanovych Shakhlin	Russia	13	7	4	2
Edoardo Mangiarotti	Italy	13	6	5	2
Takashi Ono	Japan	13	5	4	4
Paavo Johannes Nurmi	Finland	12	9	3	0
Sawao Kato	Japan	12	8	3	1
Birgit Fischer-Schmidt	Germany	12	8	4	0

Result 35

```sql
-- 4. Average age of gold medal winners by sport
SELECT
    s.sport_name,
    ROUND(AVG(p.age), 1) AS avg_gold_medalist_age,
    COUNT(p.medal) AS gold_medals,
    MIN(p.age) AS youngest_gold,
    MAX(p.age) AS oldest_gold
FROM participations p
JOIN events e ON p.event_id = e.event_id
JOIN sports s ON e.sport_id = s.sport_id
WHERE p.medal = 'Gold' AND p.age IS NOT NULL
GROUP BY s.sport_name
ORDER BY avg_gold_medalist_age DESC;
```

sport_name	avg_gold_medalist_a...	gold_medals	youngest_go...	oldest_gold
Art Competitions	41.3	48	20	63
Alpinism	38.8	16	22	57
Polo	36.0	22	21	48
Equestrianism	35.3	319	20	58
Curling	33.7	50	23	54
Shooting	32.8	400	16	64
Motorboating	31.7	7	26	46
Sailing	31.7	433	14	59
Archery	30.7	110	17	63
Bobsleigh	30.5	131	18	48
Tug-Of-War	30.2	38	19	42
Beach Volleyball	29.9	24	23	35
Triathlon	29.0	10	24	34
Fencing	28.8	591	16	50
Skeleton	27.9	9	21	39

Result 36

```sql
-- 7. Find the youngest and oldest medal winners in each sport
SELECT
    s.sport_name,
    MIN(p.age) AS youngest_medalist_age,
    MAX(p.age) AS oldest_medalist_age,
    MAX(p.age) - MIN(p.age) AS age_range,
    ROUND(AVG(p.age), 1) AS avg_medalist_age
FROM participations p
JOIN events e ON p.event_id = e.event_id
JOIN sports s ON e.sport_id = s.sport_id
WHERE p.medal IS NOT NULL AND p.age IS NOT NULL
GROUP BY s.sport_name
ORDER BY age_range DESC;
```

sport_name	youngest_medalist_a...	oldest_medalist_a...	age_range	avg_medalist_a...
Sailing	14	71	57	31.8
Art Competitions	17	73	56	42.3
Shooting	16	72	56	33.4
Archery	14	68	54	31.0
Rowing	12	58	46	25.6
Croquet	15	58	43	31.1
Equestrianism	19	61	42	35.4
Fencing	15	52	37	28.9
Curling	22	58	36	33.3
Alpinism	22	57	35	38.8
Gymnastics	10	45	35	23.4
Swimming	12	46	34	20.9
Track and Field	15	48	33	25.0
Golf	17	50	33	27.6
Figure Skating	14	45	31	24.1

Result 37

INSERT Statement Examples

```sql

    -- INSERT STATEMENTS

    -- 1. Add Future Games
    INSERT INTO games (year, season, city)
    VALUES
        (2018, 'Winter', 'PyeongChang'),
        (2020, 'Summer', 'Tokyo');

    -- 2. Insert a new athlete
    INSERT INTO athletes (athlete_id, name, sex, height, weight)
    VALUES
        ((SELECT MAX(athlete_id) + 1 FROM athletes AS a), 'Max Lee', 'M', 188, 85.0),
        ((SELECT MAX(athlete_id) + 1 FROM athletes AS a), 'Jamie Darnold', 'F', 175, 68.5);

    -- 3. Insert a new sport
    INSERT INTO sports (sport_name)
    VALUES ('Esports');

    -- 4. Insert a new event for Basketball (sport_id = 5, for example)
    INSERT INTO events (event_name, sport_id)
    VALUES ('Basketball 3x3 Mixed', 5);

    -- 5. Insert a participation with a medal
    INSERT INTO participations (athlete_id, game_id, event_id, NOC, age, medal)
    VALUES (135573, 1, 1, 'USA', 28, 'Gold');

```

UPDATE Statement Examples

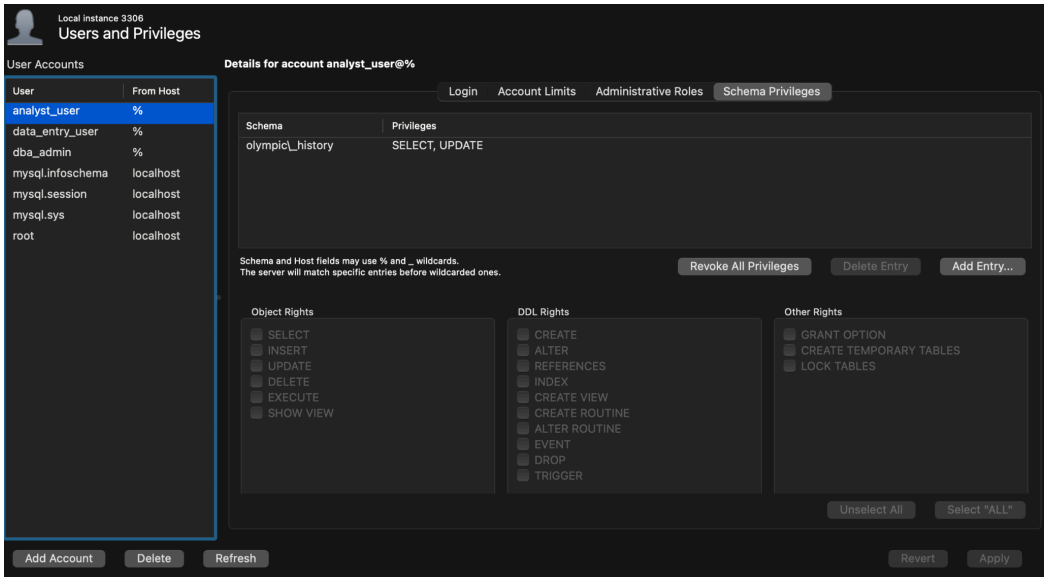
```sql
    -- UPDATE STATEMENTS

    -- 1. Update weight for a specific athlete
    UPDATE athletes
    SET weight_kg = 72.5
    WHERE name = 'Max Lee';

    -- 2. Update a country's region name
    UPDATE countries
    SET region = 'United States of America'
    WHERE NOC = 'USA';

    -- 3. Correct a city name
    UPDATE games
    SET city = 'Saint Louis'
    WHERE year = 1904 AND season = 'Summer';

    -- 4. Correct a medal type
    UPDATE participations
    SET medal = 'Gold'
    WHERE participation_id = 100;

    -- 5. Rename a sport
    UPDATE sports
    SET sport_name = 'Track and Field'
    WHERE sport_name = 'Athletics';

```

```sql
    -- 6. Correct age for a specific participation record
    UPDATE participations
    SET age = 26
    WHERE participation_id = 500;

    -- 7. Add historical notes to a country record
    UPDATE countries
    SET notes = 'Competed as West Germany before reunification'
    WHERE NOC = 'FRG';

    -- 8. Standardize all event names containing "Men's"
    UPDATE events
    SET event_name = REPLACE(event_name, "Men's", 'Mens')
    WHERE event_name LIKE "%Men's%";

    -- 9. Standardize all variations of Beijing
    UPDATE games
    SET city = 'Beijing'
    WHERE city IN ('Peking', 'Beijing', 'Bejing');

    -- 10. Correct season if it was entered wrong
    UPDATE games
    SET season = 'Winter'
    WHERE year = 1924 AND season = 'Summer' AND city = 'Chamonix';

```

DBA User Management Tasks

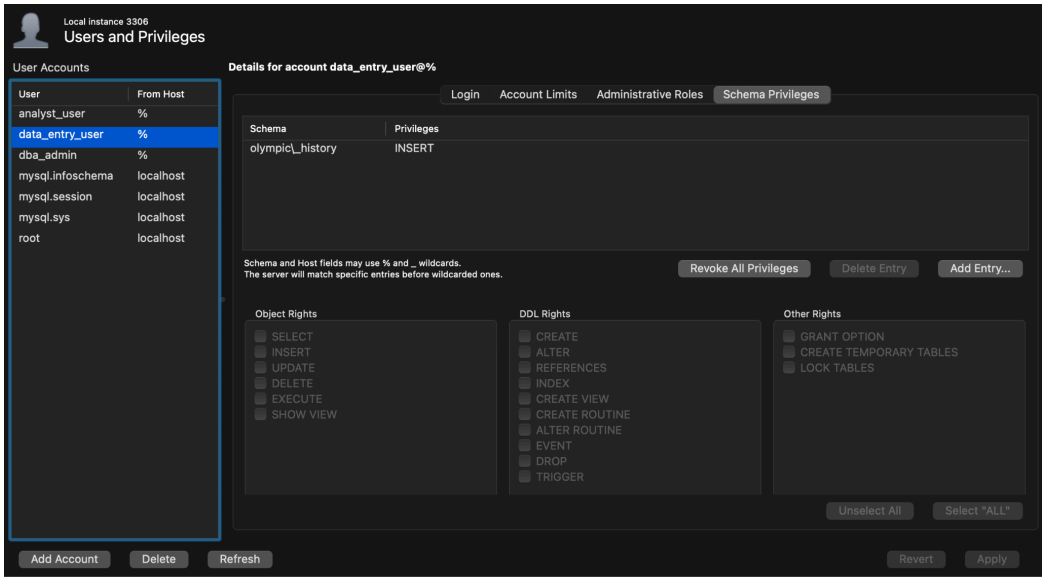
Analyst_user

User type for data analysts to read and update data. Analysts can complete their jobs with just these two rights, as they do not need to insert or delete data. Reading and updating data allow analysts to uncover insights and draw conclusions from a database.



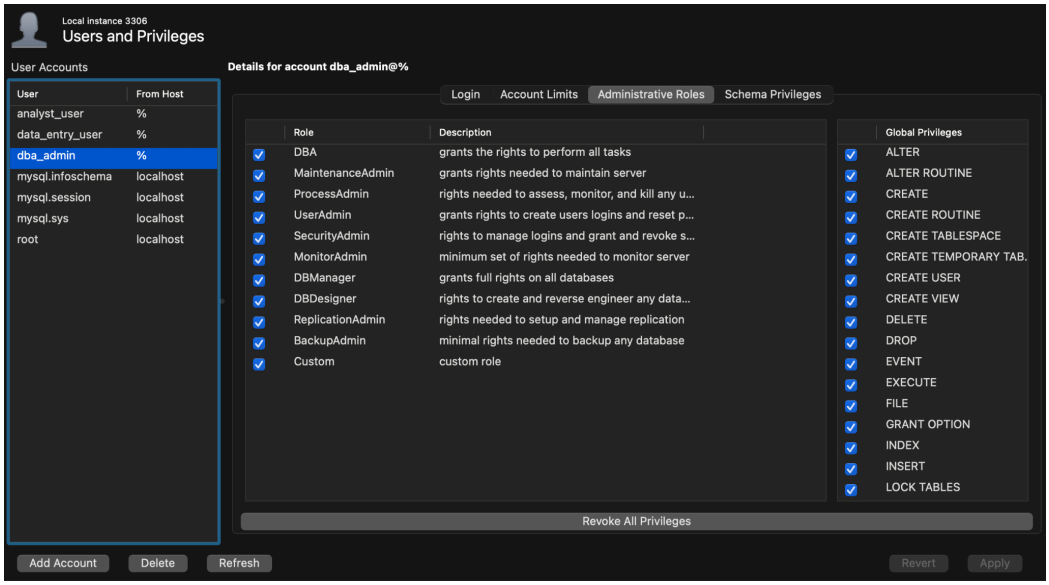
Data_entry_user

User type for data entry. Data entry only requires permission to insert new data into a database and has no need to select, update, or delete data.



Dba_admin

User type for a database admin. Administrators have complete control over database operations and are granted every permission within a database. They are the backbone for maintaining database structure and protecting the database from errors and loss.



Insights and Findings

Analysis of the 120-year Olympic history dataset revealed several significant patterns and trends through SQL queries and data exploration. The United States emerged as the leading medal-winning nation with over 4,000 total medals, followed closely by Russia and Germany, though the Winter Olympics showed more concentrated medal distribution among northern European countries due to climate and geographic factors. Michael Phelps is the clear leader in medals, with a substantial margin, having collected a total of 28 medals. The next-closest is Russian gymnast Larisa Latynina, with 18 medals. Each competition sees varying performance and participation by age, with the oldest gold medalists coming from older, retired events, such as art competitions with an average age of 41 years, or alpinism with an average of 39. However, the most extensive age range among medalists is in sailing, at 57.

Conclusion

This project successfully demonstrated the complete database development lifecycle by transforming raw Olympic history data into a fully normalized MySQL database. Through this hands-on project, I developed database-related technical skills, including ETL processes for handling missing values and data inconsistencies, ERD design for modeling complex many-to-many relationships, writing SQL queries with multiple joins and aggregate functions, and implementing role-based access control through MySQL Workbench's user management interface. The key takeaway from this project is the crucial role of thorough, upfront planning in database development. Designing the ERD and establishing business rules before implementation is essential to avoid complex and expensive restructuring. I learned that data management is an iterative process that requires both flexibility and problem-solving to transform complex, real-world data into structured, meaningful information that generates insights and supports effective decision-making.

Links

Dataset: *120 years of Olympic history: athletes and results*